

AIC-2101 Lesson Guide

Spec-Driven Development with OpenSpec



Part of
Accenture

© 2026 Ascendient, LLC

Revision 1.1.0 published on 2026-05-28

No part of this book may be reproduced or used in any form or by any electronic, mechanical, or other means, currently available or developed in the future, including photocopying, recording, digital scanning, or sharing, or in any information storage or retrieval system, without permission in writing from the publisher.

Trademark Notice: Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation without intent to infringe.

To obtain authorization for any such activities (e.g., reprint rights, translation rights), to customize this book, or for other sales inquiries, please contact:

Ascendient, LLC
1 California Street Suite 2900
San Francisco, CA 94111
<https://www.ascendientlearning.com>

USA: 1-877-517-6540, email: getinfousa@ascendientlearning.com
Canada: 1-877-812-8887 toll free, email: getinfo@ascendientlearning.com

Table of Contents

1. Spec-Driven Development with OpenSpec	6
1.1. Welcome.....	7
1.2. Who This Course Is For.....	7
1.3. What You Will Build.....	7
1.4. Course Arc.....	8
1.5. Lecture and Lab Rhythm.....	8
1.6. What You Need Today	8
1.7. How to Get the Most from Today	9
1.8. Let's Get Started	9
2. Foundations & First Cycle	10
2.1. The "Vibe Coding" Problem	11
2.2. What Happens Without Specifications	11
2.3. What Is Spec-Driven Development?	11
2.4. Prompt-Only vs. Spec-Driven	12
2.5. OpenSpec: Specs as Behavior Contracts	12
2.6. OpenSpec Works With Any Supported Agent.....	12
2.7. What Belongs in a Spec, and What Does Not	13
2.8. Core OpenSpec Workflow.....	14
2.9. What OpenSpec Creates	14
2.10. OpenSpec Directory Structure	15
2.11. When SDD Is Worth the Overhead.....	16
2.12. Context Hygiene.....	16
2.13. Reflection Questions	16
2.14. Summary	17
2.15. Additional Resources.....	17
2.16. Doesn't My Coding Agent Already Know the Tech Stack?	17
2.17. When Does This Gap Actually Matter?.....	18
3. Exploring, Proposing & Specifying.....	20
3.1. When Scope Is Unclear: The Explore Step.....	21
3.2. Updated Workflow.....	21
3.3. Anatomy of a Good Proposal	22
3.4. Feeding Context into the propose Command	22
3.5. Requirements: MUST, SHOULD, MAY	23
3.6. Scenarios: Testable Behavior Contracts	24

3.7. Delta Spec Format.....	24
3.8. What to Specify: Behavior, Not Implementation.....	25
3.9. When to Fix Outside of OpenSpec.....	26
3.10. Reviewing AI-Generated Artifacts.....	26
3.11. Reflection Questions	27
3.12. Summary.....	27
3.13. Additional Resources.....	27
3.14. How Do You Know When a Spec Is "Good Enough"?.....	28
4. Implementation Patterns & Debugging.....	29
4.1. How apply Works	30
4.2. What to Watch During Apply	30
4.3. When Changes Depend on Prior Work.....	31
4.4. Referencing Existing Specs in New Proposals	31
4.5. The Most Common "Bug" in Spec-Driven Development.....	32
4.6. Worked Example: The Tax Problem	32
4.7. Debugging Strategy: Trace Back to the Spec	33
4.8. Spec Bug vs. Code Bug	34
4.9. Not Every Cycle Needs the Same Scrutiny	34
4.10. Reflection Questions	35
4.11. Summary.....	35
4.12. Additional Resources.....	36
4.13. What If the Agent Gets Stuck?.....	36
4.14. Forcing the Agent to Ask.....	37
4.15. Tradeoffs: Ask vs. Assume	37
5. Expanded Workflow & Spec Lifecycle	39
5.1. When the Core Workflow Is Not Enough.....	40
5.2. Write the Prompt Yourself.....	40
5.3. Expanded Profile.....	41
5.4. new: Scaffold Without Generating.....	41
5.5. continue: One Artifact at a Time	42
5.6. Review Gates: What to Look For	42
5.7. ff: All Artifacts at Once.....	43
5.8. Expanded Workflow.....	44
5.9. verify: Validate Against Specs.....	45
5.10. What Verify Catches	46
5.11. Spec Suite Over Time	46
5.12. How Archive Keeps Specs Current	47
5.13. Spec Drift.....	47
5.14. Managing Spec Drift	48
5.15. When SDD Is Not Worth It (Revisited).....	48

5.16. Reflection Questions	49
5.17. Summary	49
5.18. Additional Resources.....	49
5.19. Context Management During Continue Steps.....	50
5.20. continue Is Not Just a Different Syntax.....	50
5.21. When Do You Commit? Archive and Git	51
6. Real-World Patterns & Spec Management.....	52
6.1. When Existing Behavior Changes	53
6.2. Where MODIFIED and REMOVED Go	53
6.3. MODIFIED Section	54
6.4. REMOVED Section	54
6.5. ADDED, MODIFIED, REMOVED Together	55
6.6. When SDD Is Worth It, Revisited.....	55
6.7. Judgment Spectrum.....	56
6.8. Writing Specs by Hand	56
6.9. What Makes a Hand-Written Spec Effective.....	57
6.10. Reflection Questions	57
6.11. Summary	58
6.12. Additional Resources.....	58
6.13. How the Spec Suite Becomes the System's Memory.....	58
6.14. Team Patterns for Spec Management.....	59
7. Course Wrap-Up	60
7.1. What You Built	61
7.2. What You Practiced.....	61
7.3. The Spec Suite Is the Product.....	62
7.4. The Workflow Scales.....	62
7.5. OpenSpec Is Tool-Agnostic	63
7.6. Taking This to Your Projects	63
7.7. Additional Resources.....	64

Spec-Driven Development with OpenSpec

LESSON 1

-
- ✓ Situate Spec-Driven Development in the context of modern AI-assisted software engineering
 - ✓ Navigate the course arc and the BeanBotics application you will build across the day
 - ✓ Understand the lecture/lab rhythm and what is expected of you as a learner
 - ✓ Confirm your environment is set up and you are ready to begin

1.1. Welcome

- **Course code:** AIC-2101
- **Duration:** 1 day. 6 hours of instruction, more than half of which is hands-on lab time
- **Audience:** Senior developers, AI engineers, and technical leads already working with AI coding agents
- **Format:** Short lectures paired with longer hands-on labs

1.2. Who This Course Is For

- Developers who have used AI coding agents and run into the familiar frustrations:
 - Generated code drifts from intent across sessions
 - Assumptions the agent made are invisible in the result
 - A new session means starting context from scratch
 - Code review becomes archaeology. Reconstructing intent from diffs
- Technical leads who need a reproducible, auditable approach to AI-assisted development
- Teams who want to collaborate through specifications, not just chat history

1.3. What You Will Build

- **BeanBotics:** a coffee ordering web application (Python/FastAPI + HTML/CSS/JS)
- A realistic system with real complexity: menu display, order lifecycle, customizations, receipts, cost modeling, dynamic pricing
- You start from a working skeleton and grow five feature areas through SDD
- Every lab ends with working, runnable code. Not just planning artifacts

1.4. Course Arc

Module	Focus
Foundations & First Cycle	OpenSpec overview, the core 3-command workflow, first end-to-end change
Exploring, Proposing & Specifying	Exploration before proposals, reviewing and refining AI-generated artifacts
Implementation Patterns & Debugging	Cross-feature dependencies, debugging spec ambiguity
Expanded Workflow & Spec Lifecycle	Incremental artifact building with review gates, managing spec evolution
Real-World Patterns & Spec Management	Modifying and removing specs, writing specs by hand, judgment about when to use SDD

1.5. Lecture and Lab Rhythm

- Each module: roughly 15-20 minutes of lecture, 45-50 minutes of lab
- Lectures preview what the lab covers in depth
- Labs are where the learning happens. Working code comes out of every one
- Expect more than half of your day in hands-on work

The course is deliberately lab-heavy: SDD is a discipline built through practice, not lectures. Lectures establish vocabulary and shape; labs give you the repetitions that turn technique into habit.

1.6. What You Need Today

- **VS Code** (or your preferred editor) installed
- **Node.js**: for installing OpenSpec via npm
- **Python 3.10+** with `venv`: for running the BeanBotics app

- **Git**: for checkpoint management between labs
- **An active subscription or account** for your AI coding agent

 Tip

The setup guide covers specifics. If anything is not working, flag it now. The first lab assumes your environment is ready.

1.7. How to Get the Most from Today

1. **Do every lab**: reading along is not enough; SDD is a muscle, not a concept
2. **Read the artifacts the AI generates**: do not just accept them; half the skill is critique
3. **Ask questions as they come up**: better now than after you have built on a shaky foundation
4. **Practice context hygiene**: we will clear context at the right moments throughout the day
5. **Think about your own projects**: where would SDD help, and where would it be overkill?

1.8. Let's Get Started

- Confirm your editor and your coding agent is open
- Have the BeanBotics starter repository ready
- Verify the app runs locally
- Then we dive into **Foundations & First Cycle**

Foundations & First Cycle

LESSON 2

-
- ✓ Explain why unstructured AI prompting leads to drift, hallucination, and non-reproducible results
 - ✓ Define Spec-Driven Development (SDD) and how it differs from prompt-only workflows
 - ✓ Describe OpenSpec's core 3-command workflow: *propose*, *apply*, and *archive*
 - ✓ Identify when SDD adds value and when it is overkill
 - ✓ Apply context hygiene practices when working with AI coding agents

2.1. The "Vibe Coding" Problem

- AI coding agents are powerful. They generate working code from natural language
- But conversations disappear: decisions, constraints, and intent live only in chat history
- Common symptoms of unstructured prompting:
 - **Drift**: the AI gradually moves away from the original intent
 - **Hallucination**: the AI invents requirements you never asked for
 - **Non-reproducibility**: repeating the same prompt produces different results
 - **Context loss**: starting a new session means starting from scratch
- The root cause: nothing is written down

2.2. What Happens Without Specifications

- You prompt an AI agent to add a feature
- It works, but makes assumptions about scope, error handling, and edge cases
- Next session, you (or a teammate) prompt for a related feature
- The agent has no memory of the first change, so decisions conflict
- Code review becomes archaeology: reconstructing intent from diffs

Without written specifications, three developers each prompting an AI agent independently get three different sets of assumptions. The result: inconsistent behavior, conflicting implementations, and no audit trail.

2.3. What Is Spec-Driven Development?

- **Spec-Driven Development (SDD)** means agreeing on intent, constraints, and acceptance criteria *in writing* before implementation happens
- Specifications are durable files that live in your repository alongside code
- Both humans and AI agents read from the same source of truth
- SDD is not waterfall. Specs are iterative and editable as you learn

- The core idea: **write down what you want before asking an agent to build it**

2.4. Prompt-Only vs. Spec-Driven

Aspect	Prompt-Only	Spec-Driven
Context	Lives in chat history, disappears	Durable files in the repo
Code review	Reconstruct intent from diffs	Review specs alongside code
Onboarding	Ask around, read old PRs	Read specs to understand the system
Resumption	Painful after breaks	Read specs to rehydrate
Predictability	Varies with prompt quality	Specs define expected behavior

2.5. OpenSpec: Specs as Behavior Contracts

- **OpenSpec** is a tool-agnostic framework for Spec-Driven Development
- Specifications describe *observable behavior*, not implementation details
- If the implementation can change without affecting externally observable behavior, it does not belong in the spec
- OpenSpec manages the full lifecycle: create, implement, archive

Specs live in an `openspec/` directory, organized by domain (e.g., `ordering.md`, `authentication.md`). Each contains requirements using RFC 2119 keywords (MUST, SHOULD, MAY) and testable Given/When/Then scenarios.

2.6. OpenSpec Works With Any Supported Agent

OpenSpec is tool-agnostic. This course supports seven first-class AI coding agents, grouped by how they invoke commands:

Invocation pattern	Agents	Example
Slash, colon	Claude Code	<code>/opsx:propose</code>
Slash, hyphen	Cursor, Windsurf, GitHub Copilot, Google Antigravity	<code>/opsx-propose</code>
@-mention	Amazon Q Developer, AWS Kiro	<code>@opsx-propose</code>

- All seven agents share the same `opsx` command base. Only the prefix character varies
- Each lab ships with config trees for all seven agents, so you can pick any one and the lab works
- Throughout this course, prose names the command verb (e.g., **propose**); substitute the syntax for your agent

2.7. What Belongs in a Spec, and What Does Not

Belongs in a spec (externally observable behavior):

- "The API MUST return a 404 response when the requested order does not exist"
- "The menu MUST display prices for all available sizes"
- "The system MUST retry failed database connections up to 3 times before returning an error"
- "Orders MUST NOT be placed for items that are out of stock"

Does NOT belong (internal implementation choices):

- "Use a dictionary to store orders in memory"
- "Extract the price lookup into a helper function"
- "Use SQLAlchemy instead of raw SQL"
- "Cache menu data in a module-level variable"

The test: could a different developer implement the same spec with different internal code and still pass every scenario? If yes, the spec is at the right level.

2.8. Core OpenSpec Workflow

The core workflow has three commands:

1. **propose**: define what you want to build
 - Creates a change folder with a proposal, delta specs, design, and task checklist
 - Captures the *why*, *what*, and *how* of the change
2. **apply**: implement the change
 - Works through the task checklist systematically
 - The AI agent reads the specs and builds accordingly
3. **archive**: finalize the change
 - Merges delta specs into the main specification files
 - Moves the change to an archive folder for audit trail

2.9. What OpenSpec Creates

When you invoke the **propose** command, OpenSpec generates four artifacts:

- **proposal.md**: the rationale: problem, solution, scope boundaries, risks
- **specs/**: delta specs describing what changes: ADDED, MODIFIED, REMOVED
- **design.md**: technical approach, components, tradeoffs
- **tasks.md**: implementation checklist the AI agent follows

These artifacts live in `openspec/changes/<feature-name>/` and form a complete record of the planned change.

2.10. OpenSpec Directory Structure

```

your-project/
├── openspec/
│   ├── specs/           # Source of truth -- current system behavior
│   ├── changes/         # Active work
│   │   ├── <name>/      # proposal.md, specs/, design.md, tasks.md
│   │   └── archive/     # Completed changes (timestamped)
│   └── config.yaml      # Project context (tech stack, conventions)
├── <agent config>/     # Skills and commands for your AI agent
(generated)
└── [your code]
  
```

- `specs/` : source of truth, always reflects current system behavior
- `changes/` : work queue, archive completed work to keep it clean
- `config.yaml` : injects project context into every artifact generation; developer-maintained (OpenSpec does not update it)

```

your-project/
├── openspec/
│   ├── specs/           # Source of truth -- current system behavior
│   ├── changes/         # Active work
│   │   ├── <name>/      # proposal.md, specs/, design.md, tasks.md
│   │   └── archive/     # Completed changes (timestamped)
│   └── config.yaml      # Project context (tech stack, conventions)
├── <agent config>/     # Skills and commands for your AI agent
(generated)
└── [your code]
  
```

⚠ Important

Keep `config.yaml` current as your stack evolves. If you add Redis but `config.yaml` doesn't mention it, the next **propose** cycle will generate a design that ignores Redis entirely.

2.11. When SDD Is Worth the Overhead

SDD adds value when:

- The change spans multiple files or modules
- Decisions involve tradeoffs that need documentation
- Other people will review, extend, or maintain the code
- The work is production-critical (security, performance, compliance)
- You will return to the work after a break

SDD is overkill when:

- Fixing a typo or one-line bug
- The intent is self-evident from the change
- You are prototyping something you plan to throw away

2.12. Context Hygiene

- AI agents have limited context windows. What fits in the window affects output quality
- **Context hygiene** means being intentional about what is in the agent's context when you give it a task
- Key practice: **explore in one context, propose in a fresh context**
 - Use your agent conversationally to investigate and understand
 - Then clear the context (most agents support a "new chat" or `/clear` - style action) or start a new session
 - Propose with a clean window so specs are not polluted by exploration noise
- OpenSpec's `config.yaml` injects essential project context automatically

2.13. Reflection Questions

- When has unstructured AI prompting led to drift or hallucination in your own work?
- For a recent change you made, would SDD have added value or been overkill?

- Why do specs describe *observable behavior* rather than implementation?
- What role does each of `propose` , `apply` , and `archive` play in the cycle?
- How does invoking **propose** in a fresh context produce cleaner artifacts?

2.14. Summary

In this chapter we covered:

- The "vibe coding" problem: unstructured AI prompting leads to drift, hallucination, and lost context
- Spec-Driven Development: writing specifications before implementation to align humans and AI agents
- OpenSpec: a tool-agnostic framework that manages specifications as behavior contracts
- The core 3-command workflow: **propose, apply, archive**
- When SDD is worth the overhead, and when it is not
- Context hygiene: keeping the AI agent's context window clean for better output

2.15. Additional Resources

- [OpenSpec Website](#) - Framework documentation and getting started guide
- [OpenSpec GitHub Repository](#) - Source code and issue tracker
- See your AI coding agent's documentation for installation and sign-in instructions (Claude Code, Cursor, Windsurf, GitHub Copilot, Google Antigravity, Amazon Q, or AWS Kiro)

2.16. Doesn't My Coding Agent Already Know the Tech Stack?

Yes, your agent can discover the stack by reading the code and a project-context file like `AGENTS.md` . OpenSpec does not.

Who reads what, and when:

- **Agent project-context file** (`AGENTS.md` , or an agent-specific equivalent): read by your coding agent during implementation

- `config.yaml`: read by OpenSpec during *artifact generation* (when it writes the proposal, specs, design, and tasks)

`config.yaml` is what guarantees planning artifacts reflect your real stack.

Each agent looks for project context in its own conventional location. The course's checkpoints ship a single `AGENTS.md` (an emerging cross-agent convention), so the same file informs whichever supported agent you use:

Agent	Project-context file
Claude Code	<code>AGENTS.md</code> (and <code>CLAUDE.md</code> as a compatibility shim)
Cursor	<code>AGENTS.md</code>
Windsurf	<code>AGENTS.md</code>
GitHub Copilot	<code>AGENTS.md</code>
Google Antigravity	<code>AGENTS.md</code>
Amazon Q	<code>AGENTS.md</code>
AWS Kiro	<code>AGENTS.md</code>

OpenSpec injects `config.yaml` into the prompt that generates artifacts, it does not read `AGENTS.md` or scan your codebase. If `config.yaml` says nothing about Redis, the design may invent a different caching approach even when the codebase already uses Redis.

2.17. When Does This Gap Actually Matter?

If you explore the codebase with your agent before proposing, the gap shrinks, the agent carries what it learned into the propose step.

The gap matters most when:

- You invoke **propose cold**: no prior exploration in the session
- A teammate proposes from a fresh clone with no conversation history
- The generated artifacts need to stand alone for review by someone who was not in the conversation

`config.yaml` is a safety net: it guarantees planning artifacts reflect the real stack regardless of what the agent explored beforehand.

Exploring, Proposing & Specifying

LESSON 3

-
- ✓ Explain when and why to explore a codebase before proposing a change
 - ✓ Describe the four components of a well-structured proposal
 - ✓ Write requirements using RFC 2119 keywords (MUST, SHOULD, MAY)
 - ✓ Construct testable scenarios using Given/When/Then format
 - ✓ Apply the delta spec format to describe changes as ADDED, MODIFIED, or REMOVED

3.1. When Scope Is Unclear: The Explore Step

- In the previous lab, the feature scope was handed to you: "add drink size selection"
- Real-world development rarely starts that cleanly
- Common starting point: "I think something needs to change, but I am not sure what is involved"
- The **explore step** is unstructured investigation. No artifacts, no commitments
- The **explore** command puts the AI agent into thinking-partner mode
- Explore to understand the current state, discover gaps, and clarify intent, *then* propose

3.2. Updated Workflow

In the previous lab, you used the core 3-command workflow:

propose → **apply** → **archive**

The full workflow adds an **optional** step at the beginning:

explore → **propose** → **apply** → **archive**

- **Explore** is optional. Skip it when the scope is already clear
- Explore produces *understanding*: but understanding lives in context, and context gets cleared
- Before you clear: **ask the agent to draft a proposal summary**: problem, scope, risks, key findings
- Save that draft to a file (e.g., `docs/explore-notes.md` or a scratch file)
- Then clear context and use the draft as input when you invoke the **propose** command
- This bridges explore and propose without carrying forward conversation noise

Not every feature needs exploration. The drink size feature was already scoped; this lab's order status lifecycle needs investigation first, the codebase already has clues about what is missing.

The draft proposal is a **bridge document** that survives context clearing. In a fresh context, the agent reads the draft and produces cleaner artifacts than from a cold start.

3.3. Anatomy of a Good Proposal

A proposal answers four questions in writing:

1. **What problem are we solving?** The "why": what is broken, missing, or suboptimal?
2. **What is the proposed solution?** The "what": a concise description of the change.
3. **What is in scope and out of scope?** The boundaries. Equally important to say what you will *not* do.
4. **What could go wrong?** The risks. What assumptions might be wrong? What might break?

A good proposal is **skimmable in two minutes**. If it takes longer, the change might be too big. Consider splitting it into smaller changes.

3.4. Feeding Context into the propose Command

The change name you pass to the **propose** command is a directory slug, not the proposal itself. With only a slug, the agent infers intent from the codebase:

Running propose in Claude Code

```
/opsx:propose add-order-status-lifecycle
```

For anything more than a one-sentence change, give it richer context up front with a lead-in prompt:

Running propose with a read directive in Claude Code

```
Read docs/explore-status-lifecycle.md for context on what we want to build, then /opsx:propose add-order-status-lifecycle
```

Or describe the intent inline. There is no length limit:

Running propose with inline context in Claude Code

```
/opsx:propose add-order-status-lifecycle
```

The Order model defines 5 statuses but only pending and cancelled are used. Activate the full lifecycle with forward-only transitions. Backend: PUT endpoint with validation. Frontend: status-grouped queue board. Out of scope: notifications, time estimates.

The more context you provide upfront, the less the AI has to guess, and the fewer corrections you need to make in the review step.

3.5. Requirements: MUST, SHOULD, MAY

Specifications use **RFC 2119 keywords** (Request for Comments, the standard format for internet specifications) to express obligation levels:

- **MUST / MUST NOT**: absolute requirements. The system will not work correctly without them.
- **SHOULD / SHOULD NOT**: recommended behavior. Deviations require documented justification.
- **MAY**: truly optional. The implementer decides.

These keywords remove ambiguity. Compare:

- *Vague*: "The system updates the order status"
- *Precise*: "The system **MUST** enforce forward-only status transitions: pending → preparing → ready → completed"

When the AI agent reads specs with RFC 2119 keywords, it knows which behaviors are non-negotiable and which allow flexibility.

RFC 2119 is a real IETF standard (<https://datatracker.ietf.org/doc/html/rfc2119>) widely used in technical specifications; OpenSpec adopts it for the same precision. MUST means "if not met, the implementation is incorrect"; SHOULD means "there may be valid reasons to deviate, but the implications must be understood."

3.6. Scenarios: Testable Behavior Contracts

Scenarios make requirements concrete using **Given/When/Then** format:

- **Given:** the precondition or starting state
- **When:** the action or event
- **Then:** the expected observable outcome

Good scenario

```
Given an order with status  
"pending"  
When the barista advances the  
order  
Then the order status changes to  
"preparing"  
    and the order queue shows the  
updated status
```

Weak scenario

```
Given an order exists  
When the status is updated  
Then it should work correctly
```

The difference: the good scenario has **concrete values** and **observable outcomes**. Anyone can verify it. The weak scenario is vague: "work correctly" means different things to different people.

3.7. Delta Spec Format

Rather than restating the entire specification, delta specs describe **what changes**:

- **ADDED:** new requirements this change introduces
- **MODIFIED:** changes to existing requirements (includes previous version for comparison)
- **REMOVED:** requirements being deprecated (includes the reason why)


```
## ADDED
### REQ: Order status transitions
The system MUST advance orders: pending -> preparing -> ready ->
completed.

## MODIFIED
### REQ: Order cancellation
Previous: Orders MAY be cancelled at any time.
Updated: Orders MUST only be cancellable while in "pending".
```

Reviewers see exactly what changed. No diffing the full spec.

3.8. What to Specify: Behavior, Not Implementation

Err on the side of **more** detail. Every gap in a spec is a place the AI assumes. Keep the detail at the **behavior** level, not the implementation level.

Implementation detail (constrains the AI unnecessarily):

```
The button MUST be rendered as
an HTML <button>
element with class "advance-btn"
using
addEventListener for click
handling...
```

Behavioral requirement (gives the AI room to build well):

```
The order queue MUST display an
action to advance
each non-terminal order to its
next status.
```

The behavioral version survives a frontend rewrite. The implementation version breaks.

The rule: be thorough about *what* and *when*. Be silent about *how*.

3.9. When to Fix Outside of OpenSpec

Not every issue needs a spec change. After the AI implements a feature, you may find:

- **Behavior is wrong** (wrong transition rules, missing validation). This is a **spec issue**. Fix the spec, then re-apply.
- **Behavior is correct but it looks wrong** (ugly button, poor spacing, bad colors). This is a **cosmetic issue**. Just tell the coding agent to fix it directly.

The test: **do all the scenarios in the spec still pass?** If yes, the spec is fine, ask the agent directly ("Make the advance button match the style of the existing cancel button") with no proposal or artifacts.

This is the "when SDD is overkill" principle in action: specs govern behavior; visual polish and obvious fixes are better handled conversationally.

3.10. Reviewing AI-Generated Artifacts

OpenSpec generates artifacts, but **you are the reviewer**. The AI's first draft is a starting point, not a final product.

When reviewing generated artifacts, ask:

- **Proposal:** Does the scope match my intent? Did the AI add features I did not ask for?
- **Specs:** Are scenarios concrete and testable? Do they describe behavior, not implementation?
- **Design:** Does the approach fit the existing codebase? Did the AI make incorrect assumptions?
- **Tasks:** Is each task small enough to verify independently? Are dependencies ordered correctly?

Editing artifacts *before* apply is how you steer the AI. A refined spec produces better code than a vague one, every time.

This skill develops with practice. Early labs: students accept most of what the AI generates. Later labs: they should actively edit proposals, tighten scenarios, and reorder tasks, not to rewrite everything, but to catch the gaps that would lead to incorrect implementation.

3.11. Reflection Questions

- When is exploring the codebase worth the time, and when would you propose cold?
- What distinguishes a good Given/When/Then scenario from a weak one?
- Why does every proposal need explicit "out of scope" boundaries?
- Rewrite "the system updates the order status" as a MUST requirement with concrete values.
- How do you decide whether an issue needs a spec change or a direct fix?

3.12. Summary

In this chapter we covered:

- The **explore step**: investigate before proposing when scope is unclear
- **Draft a proposal summary** at the end of exploration to bridge into the propose step
- The updated workflow: **explore** → **propose** → **apply** → **archive**
- Good proposals are **skimmable in two minutes** and define scope boundaries
- **RFC 2119 keywords** (MUST, SHOULD, MAY) remove ambiguity from requirements
- **Given/When/Then scenarios** make behavior contracts testable
- **Delta specs** describe what changes: ADDED, MODIFIED, REMOVED
- **Err on the side of more detail**: but specify behavior, not implementation
- **Review AI-generated artifacts** before applying. You are the reviewer, not just the requester

3.13. Additional Resources

- [RFC 2119: Key Words for Use in RFCs](#) - The standard behind MUST, SHOULD, and MAY
- [OpenSpec Website](#) - Framework documentation and guides
- [Gherkin Reference \(Given/When/Then\)](#) - Background on the scenario format used in specifications

3.14. How Do You Know When a Spec Is "Good Enough"?

There is no universal standard for spec quality, but there are practical signals:

Ready to apply:

- Every scenario uses concrete values (specific drink names, prices, status values)
- You can explain the scope to a colleague in under a minute
- The spec does not mention variable names, CSS classes, or function signatures
- Edge cases are addressed (what happens with invalid input? boundary conditions?)

Needs more work:

- Scenarios use words like "appropriate," "correct," or "properly" without defining them
- The scope is unclear. Would two developers draw the same boundary?
- Requirements mix behavior ("return 404") with implementation ("use a try/except block")
- You added a feature to the proposal that was not in your original intent

When in doubt, apply the *two-developer test*: could two developers implement this spec independently and produce systems that behave identically from the user's perspective? If not, the spec needs more precision.

Implementation Patterns & Debugging

LESSON 4

-
- ✓ Describe how apply executes the task checklist
 - ✓ Reference existing specs when proposing dependent changes
 - ✓ Trace unexpected behavior back to spec ambiguity
 - ✓ Debug by identifying spec gaps, adding requirements, re-applying
 - ✓ Distinguish spec bugs from code bugs when verifying implementations

4.1. How apply Works

When you invoke the **apply** command, the AI agent reads three things:

1. **tasks.md**: the ordered checklist. The agent works through tasks sequentially, checking each one off as it completes.
2. **specs/**: the behavioral requirements. The agent uses these to decide *what* the code should do.
3. **design.md**: the technical approach. The agent uses this to decide *how* to structure the code.

The agent does not improvise; it follows the plan you reviewed and refined. Clear specs produce correct implementations; vague specs produce assumptions that may not match your intent.

4.2. What to Watch During Apply

While the agent works, pay attention to how it behaves:

- **Task order:** Does it follow the sequence in **tasks.md**? If tasks have dependencies, order matters.
- **Ambiguity handling:** When the spec does not cover something, does the agent ask or assume? Most agents assume, and you may not notice until you verify.
- **Spec references:** Does the agent refer back to the specs and design, or does it just write code? Agents that drift from the plan tend to produce implementations that pass some scenarios but miss others.
- **Task completion:** Does it check off tasks as it goes? Unchecked tasks may indicate the agent got stuck or skipped something.

Watching *how* the agent works helps you identify problems early, before you find them in the running application.

Behavior varies by model, task complexity, and context-window state. Some agents ask clarifying questions when specs are ambiguous; others make a choice and move on. Knowing your agent's pattern helps you decide how much detail belongs in specs vs. left to the agent's judgment.

4.3. When Changes Depend on Prior Work

Real development is cumulative: when a new change depends on a previous one, the AI agent needs to know about that dependency.

- The **code** from prior work is in the codebase, and the agent can read it
- The **specs** capture what the code alone does not: edge cases, constraints, behavioral contracts

Example: a customization system adds \$0.75 for an extra shot. Now you are building the receipt view.

- **Code only:** the agent sees `price += 0.75`
- **Code + spec:** it also sees *"each customization MUST appear as a separate line item with its individual price"*
- The difference shows up when the receipt needs to itemize

4.4. Referencing Existing Specs in New Proposals

After you archive a change, its delta specs merge into `openspec/specs/`. The next time you propose, those specs are available as context.

Without spec reference

Invoke **propose** with only a slug. The agent reads code and `config.yaml` but misses behavioral context from prior specs. It may not respect edge cases that the code alone does not express.

With spec reference

Lead with a prompt that points at the relevant specs, then invoke **propose**:

```
Read the customization specs in
openspec/specs/.
Receipts must show each
customization as a line
item with its individual price.
```

With the spec reference, the agent has both the code *and* the behavioral contract, so generated artifacts respect prior-feature constraints.

The rule: when proposing a change that depends on prior work, point the agent at the relevant specs. The specs capture intent that the code alone does not communicate.

 Tip

Not sure which specs are relevant? Invoke **explore** to ask the agent to review `openspec/specs/` and identify dependencies before you propose.

This technique scales: when a change touches pricing, customizations, and the financial model simultaneously, referencing the relevant specs upfront saves time spent correcting generated artifacts. Think of it as briefing a new team member: you would not just point them at the code; you would also explain the requirements.

4.5. The Most Common "Bug" in Spec-Driven Development

When the AI builds something that does not match your expectations, the instinct is to look at the code. In SDD, the most common root cause is actually a **spec gap**.

- A **spec gap** is a requirement you did not write, and the agent fills it with an assumption that may not match your intent
- **This is not a failure of the AI.** It is a natural consequence of incomplete specifications
- Every gap is a place the agent decides on your behalf, with no way to know whether its choice matches yours

4.6. Worked Example: The Tax Problem

A receipt spec says:

```
The receipt MUST display the total price including tax.
```

This looks complete, but consider the questions it does not answer:

- **What is the tax rate?** 8%? 8.5%? 10%? Varies by location?
- **Per item or on the subtotal?** Tax each line item, or tax the total?
- **How do you round?** Round each line item to the nearest cent? Or round only the final total?

Each unanswered question leads to a different valid implementation. The AI picks *one* answer for each, and you may not notice until the receipt total is off by a few cents.

```
# Rounding per item vs. rounding at the end

3 items at $1.05 each, 8.5% tax:

Per-item rounding:    3 x round($1.05 x 1.085) = 3 x $1.14 = $3.42
Subtotal rounding:    round(3 x $1.05 x 1.085) = round($3.4178) =
$3.42

# Same here -- but try $1.03 at 8.5%:
Per-item rounding:    3 x round($1.03 x 1.085) = 3 x $1.12 = $3.36
Subtotal rounding:    round(3 x $1.03 x 1.085) = round($3.3527) =
$3.35
```

One cent difference. From a single unanswered question in the spec.

Tax rounding is a well-known source of bugs in financial software, but the point isn't that students need to be tax experts. Seemingly-complete specs can have hidden gaps that manifest as subtle, hard-to-trace bugs, this example will surface when students verify a test order in the lab.

4.7. Debugging Strategy: Trace Back to the Spec

When the implementation does not match your expectations:

1. **Verify the behavior.** What did you expect? What actually happened? Be specific. "the total is wrong" is not enough. "The total is \$3.36 but I calculated \$3.35" gives you something to trace.
2. **Read the spec.** Does the spec cover this case? If the spec says "display the total including tax" but does not specify the tax rate, the gap is in the spec.
3. **Identify the assumption.** What did the AI decide? It chose a tax rate, a rounding strategy, or a calculation order. Find the choice it made.
4. **Fix the spec.** Add the missing requirement with concrete values: "Tax rate MUST be 8.5%, applied to the order subtotal, rounded to the nearest cent on the final total."
5. **Re-apply.** The agent reads the updated spec and adjusts the implementation.

This is not just a debugging technique, it is **how specs improve over time**. Every gap you find and fill makes the spec more precise for the next change.

4.8. Spec Bug vs. Code Bug

Not every issue is a spec gap. Sometimes the spec is clear and the AI just wrote incorrect code. Knowing the difference saves time:

Spec bug: the spec is incomplete or ambiguous:

- The spec says "display the total including tax" but does not specify the tax rate
- The spec says "advance the order" but does not say what happens to a completed order
- Fix: edit the spec, then re-apply

Code bug: the spec is clear but the implementation is wrong:

- The spec says "tax rate MUST be 8.5%" but the code uses 0.85 instead of 0.085
- The spec says "cancel is only available for pending orders" but the cancel button appears on all orders
- Fix: point the agent at the spec and the incorrect code, ask it to fix the implementation

Cosmetic issue: the behavior is correct but it looks wrong:

- Buttons are misaligned, colors are off, spacing is inconsistent
- Fix: just tell the agent directly. No spec needed.

The diagnostic question: Does the spec cover this case with concrete, testable language? If yes, it is a code bug. If no, it is a spec bug. If the behavior is correct and the issue is visual, it is cosmetic.

4.9. Not Every Cycle Needs the Same Scrutiny

In the previous lab, you reviewed every artifact in detail. That is the right approach for complex or ambiguous features, but not every change is complex or ambiguous.

When to invest in deep review:

- The change touches multiple parts of the system
- The scope was unclear and you had to explore first
- The feature has edge cases or dependencies on prior work
- You are not confident the AI understood your intent

When a quick scan is enough:

- The change is well-scoped and you know exactly what you want
- The scenarios are straightforward with few edge cases
- You have built similar features before
- The stakes are low. A mistake is easy to catch and fix

Match scrutiny to risk: over-reviewing simple changes wastes time; under-reviewing complex ones creates bugs.

4.10. Reflection Questions

- When should you reference prior specs explicitly in a new proposal?
- What does "the most common bug is a spec gap" mean in your own words?
- Walk through the five debugging steps from unexpected behavior to fix.
- How do you tell a spec bug from a code bug?
- Which changes deserve deep review, and which deserve a quick scan?

4.11. Summary

In this chapter we covered:

- How the **apply** command reads specs, design, and tasks to drive implementation
- **Reference existing specs** when proposing changes that depend on prior work. The specs capture constraints the code alone does not communicate
- The most common "bug" in SDD is a **spec gap**, not a code bug. Every unspecified detail is a decision the AI makes for you
- **Debugging strategy:** verify the behavior, read the spec, identify the assumption, fix the spec, re-apply
- **Spec bugs vs. code bugs vs. cosmetic issues:** diagnose which one you have before deciding how to fix it

- **Match review depth to risk:** deep review for complex changes, quick scan for well-scoped ones

4.12. Additional Resources

- [OpenSpec Website](#) - Framework documentation and guides
- Your AI coding agent's official documentation - workflow integration, command syntax, and configuration

4.13. What If the Agent Gets Stuck?

Sometimes the **apply** command stalls partway through the task list. The agent may:

- Repeat the same failed approach
- Ask a question you already answered in the spec
- Produce code that does not compile or run

Strategies for unsticking the agent:

- **Narrow the task.** Break broad tasks into smaller steps in `tasks.md`.
Example: "implement the receipt view" becomes "add the receipt route" → "build the receipt template" → "wire up the price calculations."
- **Provide an example.** When the agent struggles with a described format, show a concrete input/output: "Given an order with a Large Latte (\$5.50) and extra shot (\$0.75), the receipt should look like this: ..."
- **Clear context and re-apply.** When the window is cluttered with failed attempts, start fresh. Specs persist in files. The agent re-reads them cleanly.
- **Check the design doc.** The design may conflict with the codebase. Example: design says "add a new SQLite table" but the project uses in-memory storage. Fix the design.

In each case, change the *inputs* (specs, design, tasks, context), not the code yourself. The goal is to get the agent back on track, not take over.

4.14. Forcing the Agent to Ask

Instead of discovering spec gaps after the fact, you can tell the agent to **ask** when it encounters ambiguity. OpenSpec's `config.yaml` supports per-artifact rules that are applied automatically on every generation:

```
# openspec/config.yaml
rules:
  specs:
    - When a requirement is ambiguous or underspecified,
      ask the user to clarify rather than assuming a default
    - Flag any pricing, rounding, or calculation details
      that are not explicitly stated
  tasks:
    - If a task depends on a detail not covered in the
      specs or design, stop and ask before proceeding
```

Rules in `config.yaml` persist across sessions: set them once and every invocation of **propose** and **apply** picks them up automatically.

For broader coverage, the `context` field applies to all artifact types:

```
context: |
  When you encounter ambiguity -- missing rates, unspecified
  edge cases, unclear behavior -- ask the user rather than
  assuming. Never silently pick a default for business logic.
```

4.15. Tradeoffs: Ask vs. Assume

Requiring the agent to ask is not always better. It depends on the change.

When asking helps:

- Business logic with specific values. Tax rates, pricing rules, rounding strategies
- Edge cases where a wrong guess has real consequences. What happens to in-flight orders during a status change?
- Cross-feature dependencies. Does the receipt need to show customization line items?

When asking slows you down:

- Well-scoped changes where the proposal already provides the details. If you said "+\$0.75 for an extra shot," you do not need the agent to confirm the price
- Cosmetic or structural choices the agent can reasonably decide. File organization, variable names, CSS class structure
- Early exploration where you want the agent to move fast and show you options

The practical approach: Scope the rules to ambiguity that actually hurts, "ask about business logic and calculations" is more useful than "ask about everything." A rule that triggers too often teaches the developer to ignore it.

Expanded Workflow & Spec Lifecycle

LESSON 5

-
- ✓ Identify when features are too complex for the core workflow
 - ✓ Describe the expanded commands: new, continue, ff, verify
 - ✓ Explain why incremental generation catches issues earlier than all-at-once
 - ✓ Apply review gates between artifacts to steer AI planning
 - ✓ Manage spec evolution through archive; recognize spec drift

5.1. When the Core Workflow Is Not Enough

In the previous labs, you used the core workflow (**propose** → **apply** → **archive**) and each feature fit into a single proposal prompt. The scope was clear enough that the AI could generate all four artifacts at once and get them mostly right.

But some features have too many interrelated decisions to capture in a single prompt:

- Data models that depend on each other
- Calculations that need to be precise (financial math, pricing rules)
- Cross-cutting concerns that touch multiple parts of the system
- Features where getting the proposal right requires getting the specs right first

When you cannot write a single prompt that captures everything, you need a different pace. One where you review each artifact before the next one is generated.

5.2. Write the Prompt Yourself

Consider this feature: **ingredient cost model with financial dashboard**.

- Each drink is a recipe that consumes specific quantities of base ingredients
- Small and medium drinks use one espresso shot; large drinks use two
- The financial dashboard shows revenue, cost of goods sold (COGS), and gross margin
- The dashboard updates live as orders are placed

Try writing a single **propose** prompt that captures:

- Ingredient definitions and unit costs
- Recipe mappings per drink and size (varying espresso shots, milk quantities)
- COGS calculation including customization costs (extra shot adds ingredient cost too)
- A live dashboard that aggregates revenue, COGS, and margin across orders

Even a detailed prompt will leave gaps you won't see until the specs are already written.

In the lab, students face this exact feature. If the proposal captures the wrong scope, all four artifacts inherit the mistake, the expanded workflow lets you catch the scope problem in the proposal before specs are generated.

5.3. Expanded Profile

OpenSpec has two profiles:

Profile	Commands	Best For
Core (default)	propose, explore, apply, archive	Well-scoped features you can describe in one prompt
Expanded	Core + new, continue, ff, verify	Complex features where you need to review artifacts incrementally

Switch profiles with the CLI:

```
openspec config profile    # Select "expanded"
openspec update           # Regenerate agent commands
```

After switching, your AI agent gains new commands. The core commands still work the same way, expanded adds options, it does not change the existing workflow.

5.4. new: Scaffold Without Generating

The **new** command creates the change folder structure without generating any artifacts:

Running the new command in Claude Code

```
/opsx:new add-ingredient-cost-model
```

- Creates `openspec/changes/add-ingredient-cost-model/` with empty placeholders
- No proposal, no specs, no design, no tasks, just the container

Why start empty?

- Build artifacts one at a time and review each before generating the next
- **propose** generates all four at once; **new** lets you control the pace
- **Analogy:** ordering a multi-course meal all at once vs. one course at a time after tasting the previous one

5.5. continue: One Artifact at a Time

The **continue** command generates the *next* artifact in the dependency chain:

Running continue in Claude Code

```
/opsx:continue
```

Each invocation produces one artifact. The dependency graph determines the order:

1. **Proposal:** generated first (the root)
2. **Specs:** requires the proposal
3. **Design:** requires the proposal
4. **Tasks:** requires specs and design

Between each step you review what was generated and fix issues before the next artifact inherits them. Each review is a **gate**, an opportunity to correct course before downstream artifacts compound the mistake.

5.6. Review Gates: What to Look For

Each **continue** step has specific questions worth asking:

After the proposal:

- Is the scope right, one change or should this be split?
- Are there dependencies on prior features the AI missed?
- Is anything in scope that should not be?

After the specs:

- Are calculation requirements concrete enough to implement? (Specific rates, formulas, rounding)
- Do the specs cover how prior features interact with this one?
- Could two developers implement these specs and get the same behavior?

After the design:

- Does the data model handle all the variations in the spec? (Different quantities by size, customization costs)
- Where does configuration live, code, config file, or database?
- Does the design fit the existing architecture?

After the tasks:

- Are tasks ordered by dependency? (Data model before calculations, calculations before dashboard)
- Is each task small enough to verify independently?

Students will ask these exact questions at each **continue** step in the lab. The goal: build the habit of targeted questions rather than skim-and-approve, the same judgment skill practiced in the fast-vs-thorough cycle exercise.

5.7. ff: All Artifacts at Once

The **ff** command (fast-forward) generates all remaining artifacts in one step:

Running ff in Claude Code

```
/opsx:ff
```

This is functionally similar to **propose**: the difference is that **ff** works on an existing change folder. Use it when:

- You started with **new** but realized the scope is clear enough to generate everything
- You used **continue** for the proposal, reviewed it, and now want the rest at once
- You are confident the remaining artifacts will be correct

ff is not the opposite of **continue**. They are tools for different moments:

- **continue**: when you need to review between steps
- **ff**: when the existing artifacts are solid and you want the rest generated quickly

5.8. Expanded Workflow

Putting it together, the expanded workflow is:

new → **continue** (x4, with review gates) → **apply** → **verify** → **archive**

Or, when scope becomes clear mid-process: **new** → **continue** (for proposal) → **ff** (for the rest) → **apply** → **verify** → **archive**. The core workflow still works for simple features.

Complex feature, incremental***Running the expanded workflow in Claude Code***

```

/opsx:new add-ingredient-cost-model
/opsx:continue      # proposal --
review it
/opsx:continue      # specs --
review them
/opsx:continue      # design --
review it
/opsx:continue      # tasks --
review them
/opsx:apply
/opsx:verify
/opsx:archive

```

Simple feature, still works***Running the core workflow in Claude Code***

```

/opsx:propose fix-order-timestamp
Render the order queue timestamp
as HH:MM local
time instead of the raw ISO
string. Out of scope:
other timestamp formats, tz
configuration UI.
/opsx:apply
/opsx:archive

```

The core workflow still works inside the expanded profile: use **propose** for simple features, **new** + **continue** for complex ones. Switching profiles adds options without removing the ones you already know.

5.9. verify: Validate Against Specs

After **apply** completes, **verify** checks whether the implementation matches the specifications:

Running verify in Claude Code

```

/opsx:verify

```

It evaluates three dimensions:

- **Completeness:** are all tasks finished? Are all requirements implemented? Are all scenarios covered?
- **Correctness:** does the implementation align with the spec's intent?
- **Coherence:** are design decisions reflected in the code structure?

verify is non-blocking: it surfaces issues but does not prevent you from proceeding. Think of it as an automated reviewer that catches gaps you might miss during manual testing.

5.10. What Verify Catches

verify catches **silent omissions**: requirements in the spec that were never implemented. Manual testing checks what you remember; **verify** reads *all* requirements and checks each. Less about finding bugs, more about finding things that were never built.

When to use verify

- After a complex feature with many requirements
- When the spec covers edge cases you might forget to test
- Before archiving, to catch gaps while the change is still active

When to skip verify

- Simple features with 2-3 requirements you can check by hand
- When you have already tested thoroughly and want to archive

5.11. Spec Suite Over Time

Across the earlier labs, the `openspec/specs/` directory has grown from empty to covering multiple domains:

```
openspec/specs/  
├─ drink-size-selection/  
├─ order-status-lifecycle/  
├─ order-status-ui/  
├─ drink-customizations/  
├─ drink-customizations-ui/  
├─ order-confirmation/  
└─ order-receipt/
```

Each domain spec reflects the current truth about that part of the system. When you propose a feature that touches pricing, the specs already document how pricing works, the AI reads them and builds accordingly.

Good moment to ask students to open `openspec/specs/` and scan the domain names, they built all of it. The specs are a living record of the system's behavior, not planning artifacts that served their purpose and went away.

5.12. How Archive Keeps Specs Current

When you invoke **archive**, three things happen:

1. **Delta specs merge into** `openspec/specs/`. ADDED appended, MODIFIED replaced, REMOVED deleted.
2. **The change folder moves to** `openspec/changes/archive/`. Timestamped for audit trail.
3. **The active changes directory is cleaned up.** Only in-progress work remains.

Archive is not just cleanup, it's the mechanism that keeps `openspec/specs/` accurate. Without it, the spec suite falls behind the code and loses its value as a source of truth.

5.13. Spec Drift

Spec drift is what happens when the code changes but the specs do not.

Common causes:

- A quick fix applied directly without going through OpenSpec
- A cosmetic change that subtly affects behavior
- A developer forgets to archive after implementing
- An emergency hotfix that bypasses the spec workflow

The result: `openspec/specs/` says one thing, the code does another, and the next developer (or AI agent) who reads the specs gets an inaccurate picture.

Spec drift is the SDD equivalent of documentation rot, but worse: stale docs are merely unhelpful, while stale specs actively guide AI agents toward incorrect implementations. If specs say "tax rate is 8.5%" but a hotfix changed it to 10%, the next feature that reads the spec will use the wrong rate.

5.14. Managing Spec Drift

There is no automated solution for spec drift. It requires discipline:

- **Use OpenSpec for behavioral changes.** If the fix changes externally observable behavior, run it through the spec workflow. If it is truly cosmetic or internal, skip it.
- **Archive promptly.** Do not let completed changes sit in `openspec/changes/` for days. Archive when the work is done.
- **Treat specs like code.** If you would not leave dead code in the repo, do not leave stale specs.
- **Review specs during onboarding.** When a new team member reads the specs and something does not match the running system, that is a signal to update.

The goal is not perfection, some drift is inevitable. Catch it early and fix it before it causes downstream problems.

5.15. When SDD Is Not Worth It (Revisited)

Earlier we said SDD is overkill for typos and one-line fixes. Now you have enough experience to refine that heuristic:

Use the full spec workflow when:

- The change adds or modifies behavior that other features depend on
- You need the AI agent to understand constraints from prior work
- The change involves calculations, business rules, or edge cases
- Future changes will build on this one

Skip the spec workflow when:

- The change is purely cosmetic (styling, spacing, colors)
- The fix is obvious and self-contained (a typo, a missing null check)
- You are prototyping something you expect to throw away
- The change does not affect any spec in `openspec/specs/`

The new heuristic: if the change would require updating `openspec/specs/`, use the spec workflow. If not, just make the change.

A concrete test: does this change touch a spec? If yes, run it through OpenSpec (otherwise you're creating drift). If no, just make the change, there's nothing to drift.

5.16. Reflection Questions

- What signals that a change needs the expanded workflow instead of core?
- Why does **continue** catch issues earlier than a single **propose**?
- At each review gate (proposal, specs, design, tasks), what are you looking for?
- What is spec drift, and how does archiving prevent it?
- When is SDD still overkill, even with the expanded profile available?

5.17. Summary

In this chapter we covered:

- **When the core workflow is not enough:** complex features with interrelated decisions need incremental artifact generation, not all-at-once
- **new** scaffolds a change folder without generating artifacts
- **continue** generates one artifact at a time with review gates between each step. Catching issues early, before downstream artifacts inherit them
- **ff** generates all remaining artifacts at once when scope is clear
- **verify** validates implementation against specs, catching silent omissions
- **The spec suite is a living document:** archive keeps it current, drift degrades it
- **Match the workflow to the problem:** core for simple features, expanded for complex ones

5.18. Additional Resources

- [OpenSpec Website](#) - Framework documentation and guides
- Your agent's official documentation - Refer to the docs for your chosen AI coding agent (Claude Code, Cursor, Windsurf, GitHub Copilot, Google Antigravity, Amazon Q, or AWS Kiro) for chat-panel and command-invocation details

5.19. Context Management During Continue Steps

Each **continue** step reads the existing artifacts in the change folder. This means the AI agent rehydrates context from the files, not from conversation history.

When to keep context between continue steps:

- You are reviewing artifacts and discussing changes with the agent, the conversation context has useful decisions
- The steps are happening in quick succession and the context window is not cluttered

When to clear context between continue steps:

- The context window is filling up with implementation details from a prior apply
- You edited artifacts manually and want the agent to read the updated files fresh
- The conversation has drifted into topics unrelated to the current change

Most developers keep context during `new` → `continue` → `continue` and clear before `apply`. The planning phase benefits from continuity; the implementation phase benefits from a clean start with finalized artifacts.

5.20. continue Is Not Just a Different Syntax

A common misconception: "**continue** is just **propose** broken into steps." The workflow differs in a meaningful way.

With propose:

- The AI generates all four artifacts from a single prompt
- All are informed by the same context snapshot
- A subtle scope error in the prompt propagates to all four artifacts

With new + continue:

- Each artifact is generated in sequence, *after you review and potentially edit the previous one*
- The specs are generated from the *reviewed* proposal, not the raw prompt
- The design is generated from the *reviewed* specs
- Each step compounds your corrections

This is why **continue** catches issues earlier, you are iterating on the plan *before* any code is written.

5.21. When Do You Commit? Archive and Git

OpenSpec's archive is a file-system operation: it merges specs and moves the change folder, but it does not touch git. So when should you commit?

The key invariant: **after any commit you push, `openspec/specs/` should match the code.** If you commit code changes without archiving first, the specs describe the old system while the code implements the new one, that's spec drift baked into your history.

Two patterns work well:

Archive, then commit everything at once. After **archive**, stage code + updated `openspec/specs/` + the archived change folder. One commit captures the complete unit of work, natural fit for **propose**-sized changes.

Commit incrementally during apply, archive as the final commit. For larger changes, commit as tasks complete (safety net if something goes wrong partway). When all tasks are done and verified, invoke **archive** and commit the spec merge as the final commit. Natural fit for expanded-workflow changes.

Both patterns preserve the invariant: the last commit in the sequence has code and specs in sync.

Real-World Patterns & Spec Management

LESSON 6

-
- ✓ Write MODIFIED and REMOVED sections in delta specs to express changes to existing behavior
 - ✓ Evaluate when a change warrants the full spec workflow vs. a direct fix
 - ✓ Write proposals and delta specs by hand, without AI generation
 - ✓ Describe how the spec suite serves as persistent context across sessions, developers, and tools

6.1. When Existing Behavior Changes

Every change you have built so far used **ADDED** sections in the delta specs:

- Size selection, order lifecycle, customizations, receipts, financial dashboard
- Each one introduced new behavior without changing what was already there

But real systems do not only grow. They also **change**:

- Requirements evolve
- Assumptions get invalidated
- Features get reworked

When a new feature modifies behavior that is already specified, the delta spec needs to express *what changed*, not just what was added. This is where **MODIFIED** and **REMOVED** come in.

6.2. Where MODIFIED and REMOVED Go

In previous labs, your agent generated delta spec files automatically when you invoked **propose** or **continue**. Those files lived in `openspec/changes/<change-name>/specs/` and contained ADDED sections describing new behavior. When you invoked **archive**, the ADDED requirements were merged into `openspec/specs/`, making them the new source of truth.

MODIFIED and REMOVED sections go in the same delta spec files, in the same location. The only difference is what they express:

- **ADDED**: new requirements (what you have been writing)
- **MODIFIED**: changes to existing requirements (new in this lab)
- **REMOVED**: requirements being deleted (new in this lab)

On archive, each section type is handled differently:

- **ADDED** requirements are appended to `openspec/specs/`
- **MODIFIED** requirements replace the previous version in `openspec/specs/`
- **REMOVED** requirements are deleted from `openspec/specs/`

The result: `openspec/specs/` always reflects the current truth after archive.

6.3. MODIFIED Section

When a change alters an existing requirement, the delta spec uses a **MODIFIED** section that includes the previous version for comparison:

```
## MODIFIED

### Requirement: Menu price display (updated)
Previous: The menu MUST display the base price for each drink
at the selected size.
Updated: The menu MUST display the base price for each drink
at the selected size. During off-peak hours, the menu MUST
also show the discounted price alongside the original price
(displayed with strikethrough formatting).
```

Previous records what the spec said before; **Updated** is the new requirement. Code review is straightforward: reviewers see exactly what changed.

On archive, the MODIFIED requirement replaces the previous version in `openspec/specs/`. The previous text is preserved only in the archived change folder for audit trail.

MODIFIED distinguishes delta specs from rewriting the full spec: instead of diffing old and new versions, the change is explicit. Especially valuable when one change modifies requirements across multiple domain specs.

6.4. REMOVED Section

When a change eliminates a requirement or invalidates an assumption, the delta spec uses a **REMOVED** section with a reason:

```
## REMOVED

### Assumption: Fixed pricing per size
Reason: Drink prices are no longer fixed per size. Prices now
vary by time of day, full price during peak hours, 20% off
during off-peak hours. The base size price in menu.json
represents the full (peak) price.
```

The **Reason** is required: it prevents accidental removal by forcing every deletion to be justified. On archive, the removed requirement is deleted from `openspec/specs/`.

6.5. ADDED, MODIFIED, REMOVED Together

A single change can use all three sections. The dynamic pricing feature in the lab involves:

- **ADDED:** Pricing tier definitions (peak windows, off-peak discount percentage, time simulation slider)
- **MODIFIED:** Menu display, order placement, receipts, financial dashboard. All change behavior
- **REMOVED:** The assumption that drink prices are always the base size price

This is the first time you will write a change that modifies existing behavior across multiple domain specs. Previous labs introduced new domains; this lab reaches back into domains you built earlier and changes how they work.

This is the heart of spec management: a growing system doesn't just accumulate new specs, it evolves existing ones. MODIFIED and REMOVED are the mechanism for expressing that evolution without rewriting entire spec files.

6.6. When SDD Is Worth It, Revisited

Earlier we said SDD is overkill for typos, one-line fixes, and trivial changes. You now have enough experience to test that heuristic against.

Think about the changes you have built:

- **Drink size selection:** clear scope, straightforward. Core workflow, light review. Good fit for SDD.
- **Order lifecycle:** needed exploration first. Core workflow with deeper review. Good fit for SDD.
- **Drink customizations:** well-scoped, fast cycle. Quick scan of artifacts was enough. Good fit.
- **Ingredient cost model:** complex, interrelated. Expanded workflow with review gates. Needed SDD.

Now consider: would you use SDD to fix a button that says "Candle" instead of "Cancel"?

6.7. Judgment Spectrum

The question is not "SDD or no SDD." It is a spectrum:

Change	Approach	Why
Fix a typo in a button label	Just tell the agent	Self-evident, no spec affected
Adjust padding on menu cards	Just tell the agent	Cosmetic, no behavior change
Change the tax rate from 8.5% to 10%	Quick spec update + fix	Behavior changes, spec must stay in sync
Add off-peak discounts that modify pricing, receipts, and the dashboard	Full spec workflow (MODIFIED/REMOVED)	Cross-cutting behavioral change
Rebuild the order model from scratch	Expanded workflow with review gates	Complex, many interrelated decisions

The refined heuristic: if the change would require updating `openspec/specs/`, use the spec workflow. If not, just make the change. The spec suite is the line between "needs a spec" and "just do it."

The heuristic is testable: look at the spec suite and ask "does this change affect any of these specs?" Yes → OpenSpec; no → handle directly. This replaces the vaguer "when SDD is overkill" guidance with a concrete decision rule students can apply immediately.

6.8. Writing Specs by Hand

In every lab so far, the AI generated the delta specs and you reviewed them, but specs are not AI-only artifacts. They are documents that humans write, read, and maintain.

In the lab, you will write a proposal and delta specs **by hand**: then have the AI review your work. This reverses the usual dynamic:

- **Previous labs:** AI writes, you review
- **This lab:** You write, AI reviews

Writing a spec forces you to confront every ambiguity directly, what exactly changes about the receipt? What stays the same? What is the rounding strategy for the discount?

6.9. What Makes a Hand-Written Spec Effective

When you write delta specs by hand, apply the same standards you used when reviewing AI-generated ones:

- **MODIFIED sections include the previous version.** If you do not know what the previous requirement said, read it in `openspec/specs/` before writing the update.
- **REMOVED sections include a reason.** Never delete a requirement without saying why.
- **Scenarios use concrete values.** "The menu shows discounted prices" is vague. "The menu shows \$5.20 (was \$6.50) for a Large Neural Network Latte during off-peak" is testable.
- **Requirements use RFC 2119 keywords.** MUST, SHOULD, MAY. Not "will" or "should probably."
- **Edge cases are addressed.** What happens at the boundary between peak and off-peak? What about customization prices?

After writing, ask the AI to review: "Read my specs and identify gaps, ambiguities, or missing scenarios." The AI becomes your reviewer, the same role you played in previous labs.

6.10. Reflection Questions

- When do you use MODIFIED vs. ADDED in a delta spec?
- Why must every REMOVED requirement include a reason?
- How does archive keep `openspec/specs/` in sync with the code?
- What does the spec suite give you that code alone does not?

- How do MODIFIED and REMOVED sections change the experience of PR review?

6.11. Summary

In this chapter we covered:

- **MODIFIED** sections express changes to existing requirements. Include the previous version for reviewability
- **REMOVED** sections express deleted requirements. Always include a reason
- A single change can use **ADDED, MODIFIED, and REMOVED** together when it introduces new behavior while changing existing behavior
- **The judgment spectrum:** match the approach to the change. From "just tell the agent" for cosmetic fixes to full expanded workflow for complex cross-cutting changes
- **The refined heuristic:** if the change would require updating `openspec/specs/`, use the spec workflow
- **Writing specs by hand** is a skill. Reversing the usual AI-writes-you-review dynamic builds deeper understanding

6.12. Additional Resources

- [OpenSpec Website](#) - Framework documentation and guides
- Your AI coding agent's official documentation - command syntax and configuration details

6.13. How the Spec Suite Becomes the System's Memory

By now, `openspec/specs/` contains behavioral contracts for every BeanBotics domain. It was built incrementally as part of development, not written as after-the-fact documentation.

What the spec suite gives you that code alone does not:

- **Intent:** The code shows what the system *does*; the specs show what it was *meant* to do. When they diverge, the spec tells you which is wrong.

- **Decisions:** The archive records *why* each change was made, proposal for the problem, design for the approach, specs for the behavior.
- **Resumption:** After a break, the specs let you rehydrate. Read the spec suite for current state, the archive for history.
- **Onboarding:** A new developer reads the spec suite to understand what the system does; the archive shows how it evolved.

This is the payoff for the discipline of writing and archiving specs throughout the course.

The spec suite is not an end-of-project deliverable. It's a living artifact that grows with the system and serves as its persistent memory.

6.14. Team Patterns for Spec Management

The course focused on solo development (one developer, one agent), but the patterns extend to teams:

- **Specs as handoff surfaces.** Archived specs bridge the gap when one developer picks up a dependent feature. No need to ask "what did you build and how does it work?"
- **Spec review in PRs.** MODIFIED and REMOVED sections show *what* behavior is changing; code diffs only show *how* it changed.
- **Async collaboration.** Specs persist in the repo. Team members in different time zones review proposals and specs asynchronously, with no synchronous meetings required to align on intent.
- **Drift detection during onboarding.** When a new team member reads the specs and something does not match the running system, they have found spec drift. A feature, not a bug. It surfaces maintenance debt.

Course Wrap-Up

LESSON 7

-
- ✓ Apply the propose-apply-archive cycle to changes in your own projects
 - ✓ Differentiate between OpenSpec's core and expanded workflows based on change complexity
 - ✓ Use review gates and MODIFIED/REMOVED delta specs for multi-step changes to existing behavior
 - ✓ Identify when Spec-Driven Development adds value and when it is overkill
 - ✓ Recognize the spec suite as durable team memory that outlives any single session

7.1. What You Built

You built BeanBotics, a complete coffee ordering system, using Spec-Driven Development. Starting from a simple menu and checkout flow, you incrementally added:

- **Drink size selection**, your first propose-apply-archive cycle
- **Order status lifecycle**, placing, brewing, ready, picked up
- **Customizations and receipts**, cross-feature work spanning menu and checkout domains
- **Ingredient cost model and financial dashboard**, complex features built with incremental review
- **Dynamic pricing**, including modifications to existing specs with MODIFIED and REMOVED deltas

Along the way, you made a judgment call about when *not* to use SDD, and that judgment is just as important as knowing how.

7.2. What You Practiced

Skill	When To Use It
Writing proposals that scope a change	Every change
Reviewing and editing AI-generated specs	After the agent drafts artifacts
Writing specs by hand	Small changes or when you need tight control
Using Given/When/Then scenarios	All proposals and specs
Referencing existing specs in new proposals	Cross-feature changes
Debugging spec ambiguity (the bug was in the spec, not the code)	When implementation diverges from intent
	Every change

Skill	When To Use It
Choosing core vs. expanded workflow based on complexity	
Using review gates between artifacts	Expanded workflow
Writing MODIFIED and REMOVED delta specs	Changing existing behavior
Deciding when SDD is overkill	Before every proposal
Context hygiene, clearing context at the right moments	Between unrelated tasks

7.3. The Spec Suite Is the Product

The code you wrote is one output. The spec suite is the other.

Your `openspec/specs/` directory contains 14 behavioral contracts across 6 domains. Your `openspec/changes/archive/` contains 6 archived changes documenting the system's complete evolution. A new developer joining this project could:

- Read the spec suite to understand *what* the system does
- Browse the archive to understand *how* it evolved
- Use the specs as context for their next change

This is the payoff of Spec-Driven Development: **durable, structured context that outlives any single session.**

7.4. The Workflow Scales

You used two workflow patterns today, matched to the complexity of the work:

Core workflow, for changes you can describe in one prompt:

```
propose → apply → archive
```

Expanded workflow, for changes that need incremental review:

```
new → continue (repeat) → apply → verify → archive
```

And for trivial fixes, just talk to the agent. No spec needed.

The workflow scales from "just fix it" to full expanded workflow with review gates. The judgment call is yours: **if the change would require updating** `openspec/specs/`, **use the spec workflow. If not, just make the change.**

7.5. OpenSpec Is Tool-Agnostic

Whichever agent you picked today, the concepts transfer directly:

- **OpenSpec** works with 24+ AI coding tools, including the seven this course supports (Claude Code, Cursor, Windsurf, GitHub Copilot, Google Antigravity, Amazon Q, AWS Kiro)
- **Command invocation** uses agent-specific syntax, `/opsx:propose`, `/opsx-propose`, or `@opsx-propose` depending on the agent, but the workflow is identical
- **The spec files are plain Markdown**, any tool that reads Markdown can use them
- **AGENTS.md** provides project context for any agent that reads it, an emerging cross-agent convention

The specs you wrote today would work with any AI coding agent. The framework is the discipline, not the tool.

7.6. Taking This to Your Projects

Starting SDD on a real project:

1. **Install OpenSpec:** `npm install -g @fission-ai/openspec@latest`
2. **Initialize:** `cd your-project && openspec init`
3. **Configure:** Add your tech stack and conventions to `openspec/config.yaml`
4. **Start with the core workflow.** Use propose-apply-archive for your first few changes. Switch to expanded when you hit a feature that needs it.
5. **Build the spec suite incrementally.** No need to retroactively spec your codebase; each change you run through OpenSpec adds to the suite, and the domains that change most become best-documented.

 Tip

OpenSpec's sweet spot is multi-session features, complex changes to existing codebases, and team collaboration. Start there, do not try to spec everything on day one.

7.7. Additional Resources

- [OpenSpec Website](#), Framework documentation and guides
- [OpenSpec GitHub Repository](#), Source code, issues, and discussions
- Your agent's official documentation, for invocation syntax, context-window limits, and tool-specific features